Autonomous AI Research Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrollment Number(s):

Sr no	Student Name	Enrollment Number
1.	ANIKET JAT	EN22CS304010
2.	HIMANSHU SAHRAWAT	EN22CS404029
3.	SHRIYANSH GUHA	EN22CS303054
4.	ROUNAK PATHEKAR	EN22CS303046
5.	YASHASVI BAKORE	EN22CS303059

Group Name: Group 03D11

Project Number: AAI-29

Industry Mentor Name: Mr.Ashwin

University Mentor Name: Prof. Nishant Shrivastava

Academic Year: 2025-26

Problem Statement & Objectives

1. Problem Statement
2. Project Objectives
3. Scope of the Project

Proposed Solution

1. Key features
2. Overall Architecture / Workflow
3. Tools & Technologies Used

Results & Output

Conclusion

Future Scope & Enhancements

Problem Statement & Objectives

1. Problem Statement

In today's digital environment, users depend on search engines to gather information from various sources. However, extracting useful insights from multiple web results requires significant manual effort, including reading, filtering, and summarizing content. Traditional search systems provide links but do not generate concise, context-aware information.

This leads to inefficiency and increased time consumption. Therefore, there is a need for an intelligent system that can automatically search relevant information, understand context, and generate meaningful summaries for users.

2. Project Objectives

The main objectives of this project are:

- To develop an AI-based system for automated research
- To integrate real-time web search using an API
- To process and analyze retrieved data
- To generate concise summaries using a language model
- To design an interactive and user-friendly interface
- To reduce manual effort in searching and understanding information

3. Scope of the Project

The scope of the project includes:

- Accepting user queries as input
- Performing real-time web search using Tavily API
- Processing textual data from search results
- Generating summaries using a transformer-based model
- Displaying results through a web interface

The project does not include:

- Multi-user authentication system
- Persistent database storage
- Advanced conversational memory

Proposed Solution

1. Key Features

- Real-time web search using Tavily API
- Automated data collection and processing
- AI-based summarization using FLAN-T5 model
- Interactive web interface using Streamlit
- Automatic report generation in text format
- Simple and efficient workflow

2. Overall Architecture / Workflow

The system follows a simple sequential workflow.

First, the user provides a topic as input through the interface. The system then sends a request to the Tavily API to fetch relevant search results. The retrieved content is combined and processed. This processed data is passed to a pre-trained language model (FLAN-T5), which generates a concise summary. Finally, the summary is displayed to the user and saved as a report file.

3. Tools & Technologies Used

- **Programming Language:** Python
- **Frontend Framework:** Streamlit
- **API Used:** Tavily API
- **AI Model:** Google FLAN-T5 (via HuggingFace Transformers)
- **Libraries Used:**
    - transformers
    - python-dotenv
    - os

Results & Output

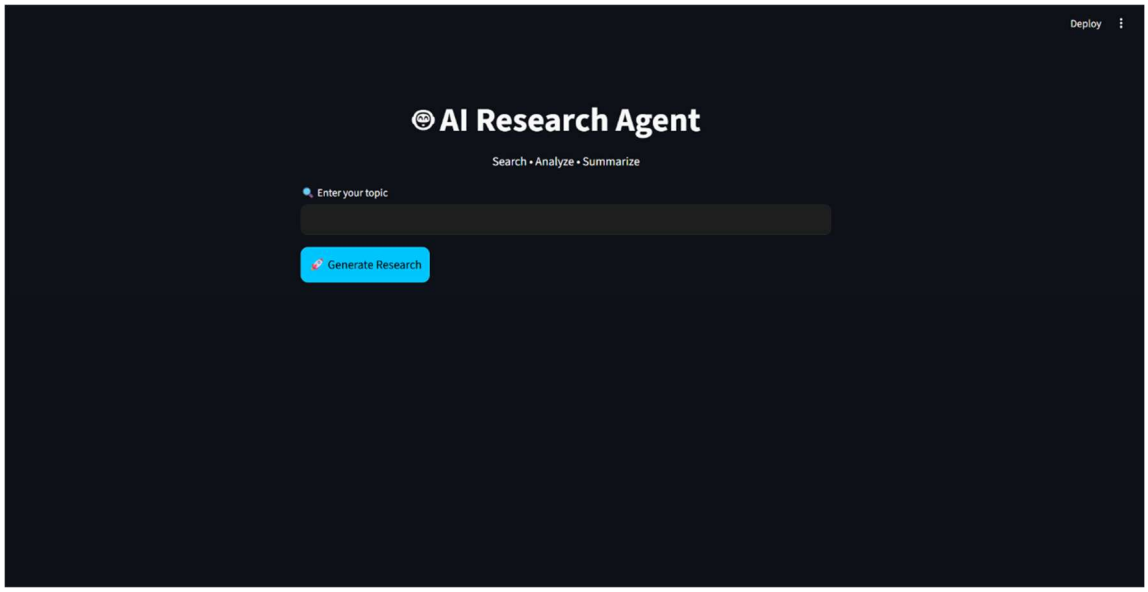
1. Screenshots / Outputs

The system successfully provides an interactive interface where users can enter a topic and generate summarized research content.

The output includes:

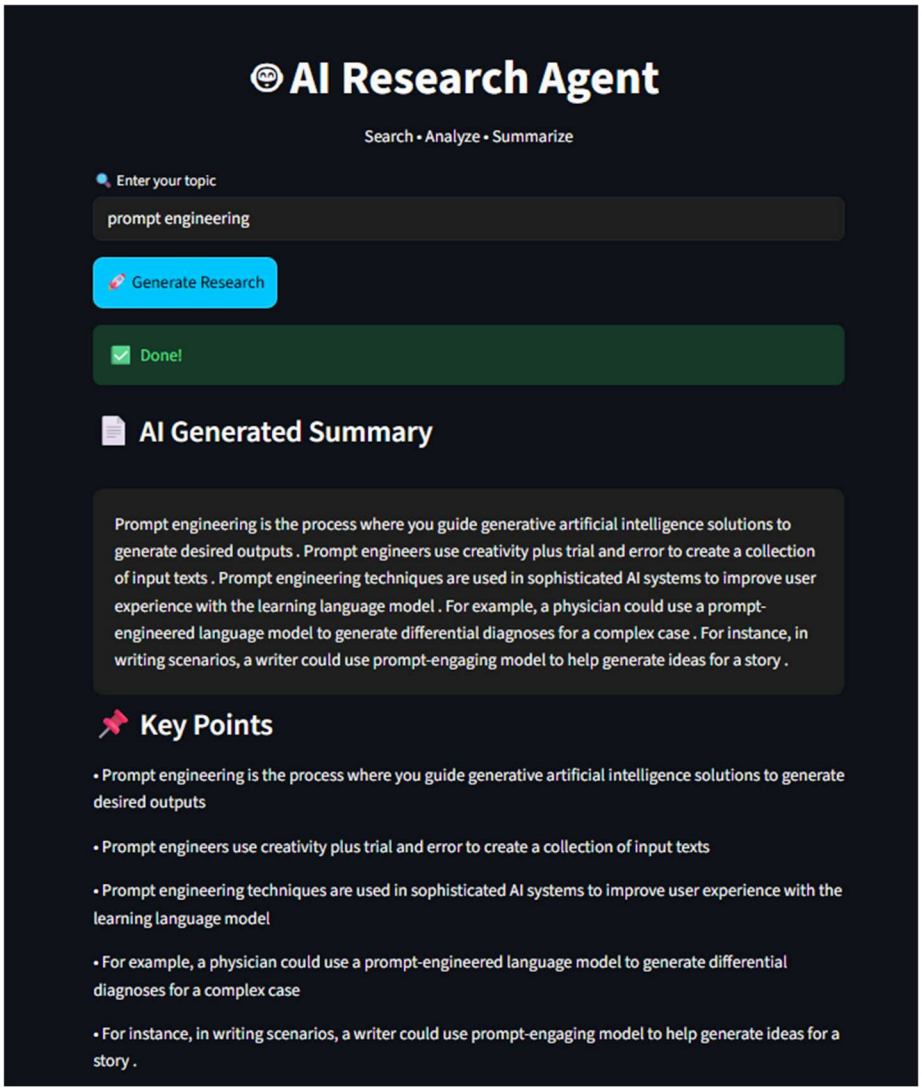
- A user interface for entering topics
- A button to trigger research generation
- Display of summarized results
- Saving the result into a text file

Figure 1: User Interface of AI Research Agent



This figure shows the front-end interface of the AI Research Agent developed using Streamlit. The system provides a clean and user-friendly layout where users can input a topic and initiate the research process. The interface is designed to simplify interaction and improve usability.

Figure 2: Execution and Output Generation of AI Research Agent



This figure illustrates the execution of the AI Research Agent, where a user query is processed and a summarized output is generated. The system performs web search, extracts relevant content, and produces a concise summary using a pre-trained language model, demonstrating end-to-end functionality.

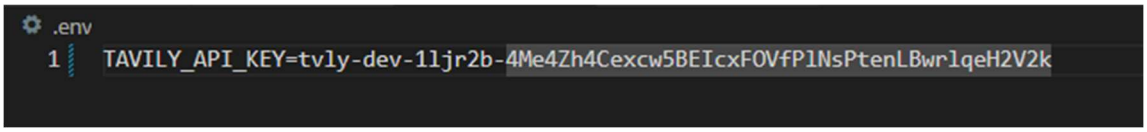
Figure 3: Backend Processing and Summarization Logic

main.py > ...

```python
from tavily import TavilyClient
import os
from dotenv import load_dotenv
from transformers import pipeline

load_dotenv()

# Tavily setup
tavily = TavilyClient(api_key=os.getenv("TAVILY_API_KEY"))

# ✅ FIXED: correct free model
summarizer = pipeline("text-generation", model="google/flan-t5-base")

# Input
query = input("Enter topic: ")

# Step 1: Search
response = tavily.search(query=query, max_results=3)
data = " ".join([r["content"] for r in response["results"]])

# Step 2: Summarize (FIXED)
summary = summarizer(
    "Summarize this: " + data[:1000],
    max_new_tokens=150,
    do_sample=False
)

result = summary[0]['generated_text']

# Remove "Summarize this:" if present
result = result.replace("Summarize this:", "").strip()
# Step 3: Save
os.makedirs("output", exist_ok=True)  # folder auto create
with open("output/report.txt", "w", encoding="utf-8") as f:
    f.write(result)

print("\n✅ Result:\n")
print(result)
```

This figure shows the backend logic of the system, including data retrieval using Tavily API and text summarization using the Hugging Face Transformers library. The system processes raw textual data and generates meaningful summaries efficiently.

Figure 4: Environment Configuration for API Integration



This figure presents the environment configuration file used for securely managing API keys. Sensitive information such as the Tavily API key is stored in the .env file and accessed using environment variables, ensuring better security and maintainability.

2. Reports / Output Details

- The input query is processed through Tavily API
- Relevant content is extracted from search results
- The extracted content is summarized using an AI model
- The final output is generated as a clean paragraph
- The result is stored in a file named report.txt

3. Key Outcomes

- Successfully developed an autonomous AI research system
- Integrated API with AI model for real-world use case
- Automated the research and summarization process
- Reduced manual effort and improved efficiency
- Demonstrated practical use of Natural Language Processing

Conclusion

This project successfully implements an Agentic AI system capable of performing automated research tasks. By integrating real-time search APIs with a transformer-based language model, the system generates accurate and concise summaries.

The project demonstrates the practical application of modern AI technologies such as API integration, natural language processing, and transformer models. It provides an efficient solution for extracting meaningful insights from large amounts of information.

Future Scope & Enhancements

The system can be further improved with the following enhancements:

- Integration of advanced language models (GPT, Gemini)
- Implementation of database storage for history
- Addition of chatbot-based interaction
- Deployment on cloud platforms
- Support for multiple queries and batch processing
- Enhanced user interface and visualization
- Addition of authentication and user management